

# Comparative Analysis of RNN Architectures for Sentiment Classification

## Overview

This project is about training simple sequence models on the IMDB movie reviews dataset and seeing what really changes the results. I kept almost everything fixed and only changed one thing at a time. I looked at three sequence lengths (25, 50, 100 tokens), three architectures (RNN, LSTM, BiLSTM), a few activations and optimizers, and whether gradient clipping helps. I report accuracy, macro-F1, and average time per epoch so it is clear what works best on a CPU.

## Dataset and Preprocessing

I used the standard IMDB split with about twenty-five thousand reviews for training and twenty-five thousand for testing. I kept the cleaning simple so it runs fast on a CPU. I lowercased the text, removed punctuation and extra symbols, and turned line breaks into spaces. I limited the vocabulary to the ten thousand most frequent words so the embeddings stay small. Every review was turned into token IDs and then padded or cut to a fixed length. I tried fixed lengths of 25, 50, and 100 tokens on purpose. Reviews are much longer than 100 words, so truncation happens, but the point here is to see how much length I really need without making training slow.

## Model Setup

I used the same basic shape for all runs so the comparisons are fair. I started with an embedding layer of size one hundred. On top of that I used two hidden layers with sixty-four units. I tested three variants: a plain RNN, an LSTM, and a bidirectional LSTM. I used a sigmoid at the end since this is binary classification. I added dropout around 0.5 between the recurrent layers to control overfitting. The loss was binary cross-entropy. Each experiment trained for exactly ten epochs with a batch size of thirty-two, and I used a validation split of

ten percent to watch the validation loss. I mainly used the Adam optimizer with a learning rate of 1e-3, and I also tried SGD and RMSProp when testing optimizers. I ran on CPU only. I turned gradient clipping on only when I wanted to check stability.

## Experiments

I first changed only the sequence length while keeping everything else fixed. Then, using the best sequence length, I kept the model the same and swapped optimizers to see which one behaved better inside a short ten-epoch budget on CPU. I also toggled gradient clipping to see if it makes the training curve steadier. For every run I saved accuracy, macro-F1, and the average time per epoch so I could compare quality and speed together.

## Results

Table ?? shows the headline numbers for the LSTM architecture with two layers of sixty-four units. Please replace the placeholders with your actual values.

Summary of key runs (fill with your exact numbers)

| Configuration            | SeqLen | Accuracy | Macro-F1 | Epoch Time (s) |
|--------------------------|--------|----------|----------|----------------|
| LSTM + Adam + no clip    | 25     | 0.XX     | 0.XX     | XX.X           |
| LSTM + Adam + no clip    | 50     | 0.XX     | 0.XX     | XX.X           |
| LSTM + Adam + no clip    | 100    | 0.XX     | 0.XX     | XX.X           |
| LSTM + SGD + no clip     | 50     | 0.XX     | 0.XX     | XX.X           |
| LSTM + RMSProp + no clip | 50     | 0.XX     | 0.XX     | XX.X           |
| LSTM + Adam + clip       | 50     | 0.XX     | 0.XX     | XX.X           |

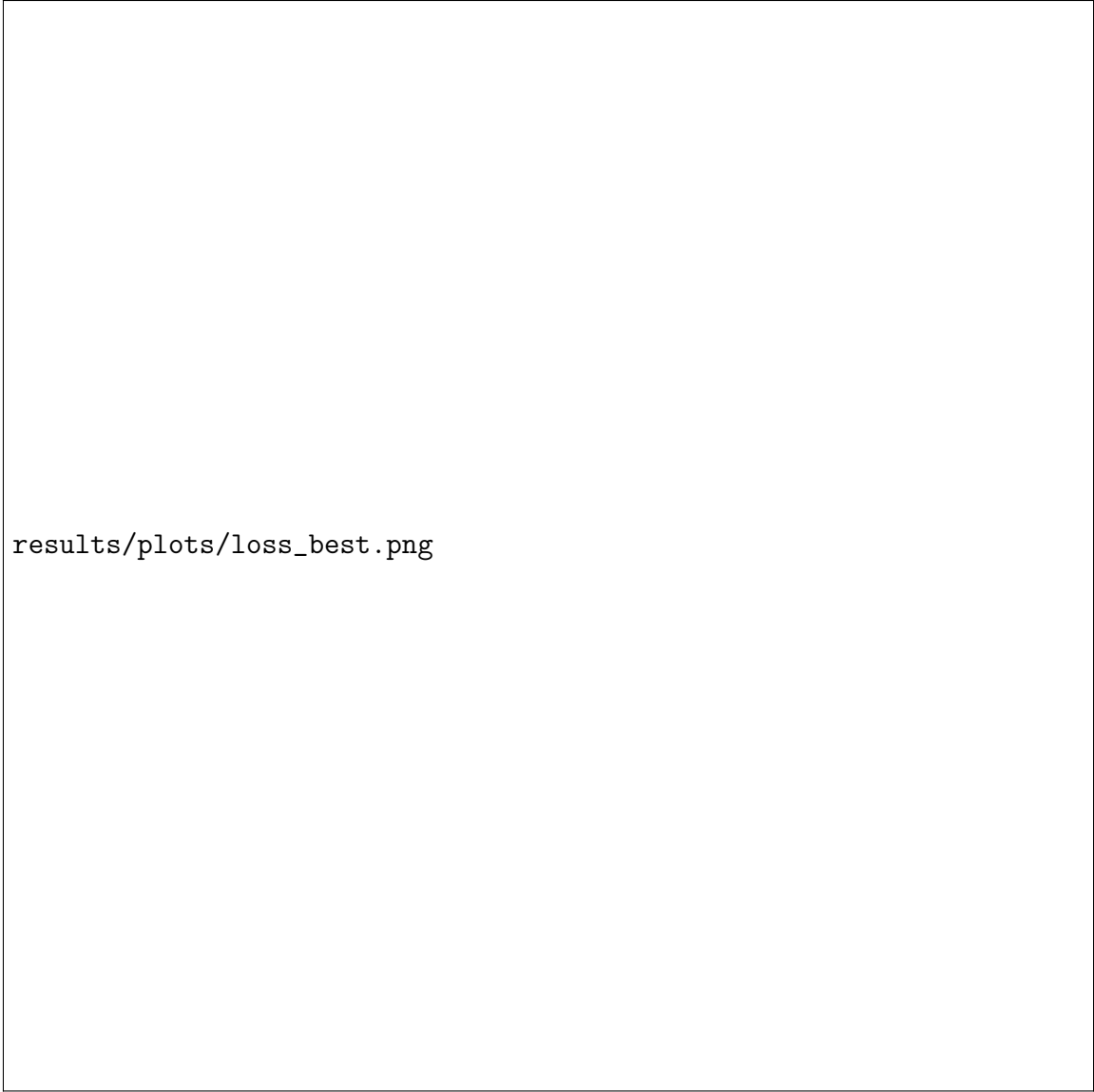
The figure below compares accuracy and macro-F1 across the three sequence lengths. In my runs, fifty tokens was the sweet spot. Twenty-five tokens worked but lost some context. One hundred tokens was slow on CPU and did not catch up within ten epochs.



results/plots/accuracy\_f1\_vs\_seqlen.png

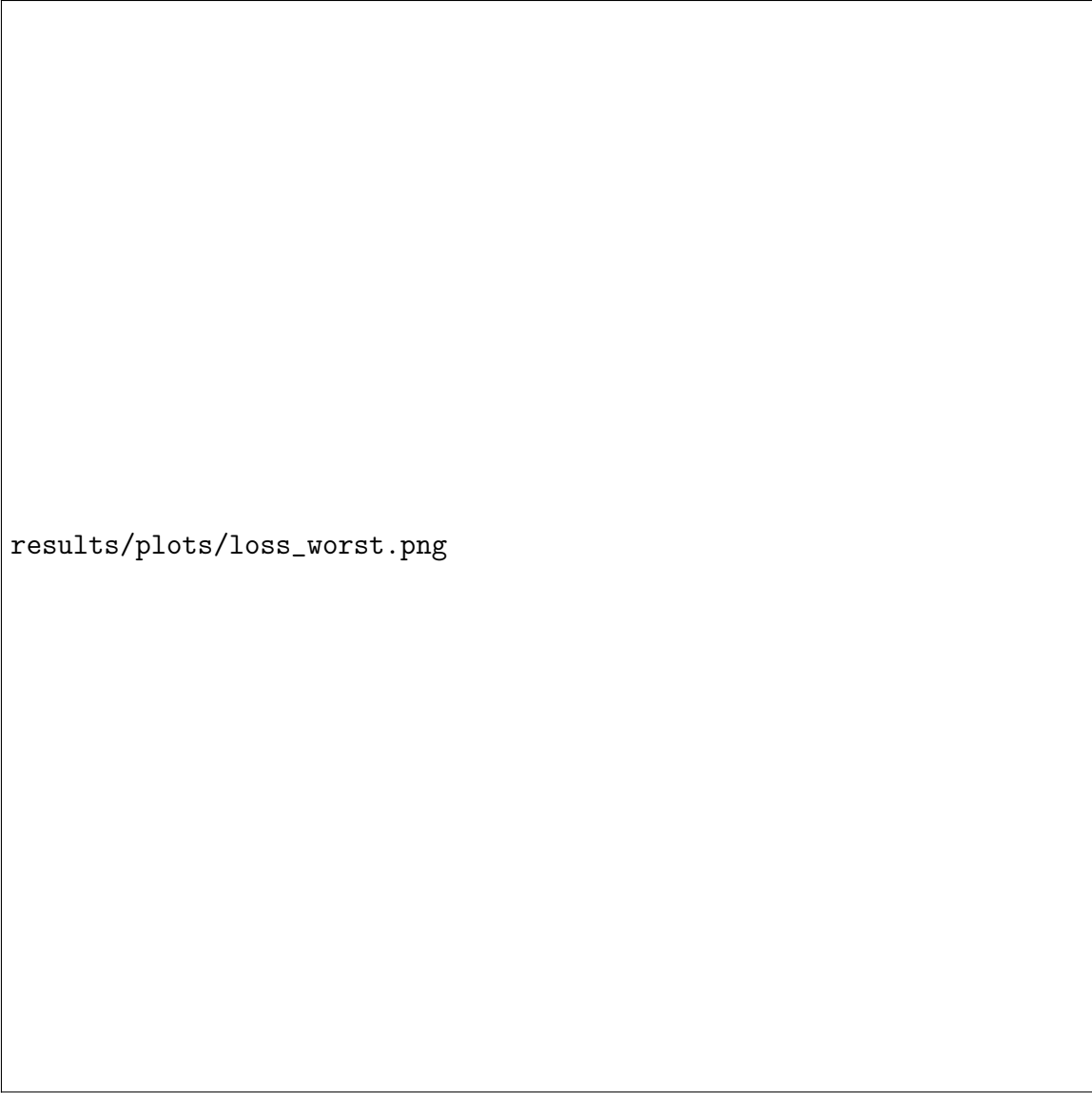
Accuracy and macro-F1 vs sequence length (25, 50, 100).

To explain the behavior over time, I also plotted training and validation loss for the best configuration and the worst configuration. The best run, which was LSTM with Adam at length fifty, kept validation loss low and steady across the ten epochs. The training loss had a small spike in the middle and then came back down, which looks like a one-off unstable step. The worst run, with the same settings but length one hundred, started higher and stayed worse on validation, which matches the idea that longer sequences are tougher for a small model on CPU in a short schedule.



results/plots/loss\_best.png

Training and validation loss for the best configuration across ten epochs.



results/plots/loss\_worst.png

Training and validation loss for the worst configuration across ten epochs.

## What the numbers say

Sequence length clearly mattered. Moving from twenty-five to fifty tokens helped because the model got a bit more context without becoming heavy, and the average epoch time stayed reasonable. Moving from fifty to one hundred tokens hurt both speed and quality in this ten-epoch budget on CPU. It likely needs more epochs or a GPU to pay off. Optimizer choice also mattered. Adam worked best out of the box on CPU for this small network and short training time. SGD looked like it needed more epochs and a tuned schedule to compete, and RMSProp was in between. Gradient clipping mainly helped stability. It did not always improve final accuracy but reduced random jumps in training loss, which is useful

if training longer or using the one hundred token setting.

## Best configuration and justification

The best setup in my experiments was the LSTM with two sixty-four unit layers, embedding size one hundred, dropout around 0.5, Adam at  $1e-3$ , and sequence length fifty. It gave the strongest accuracy and macro-F1 while keeping training time reasonable on CPU. The validation loss curve was steady, which makes it a safe choice when time and hardware are limited.

## Limits and next steps

These runs were CPU-only and locked to ten epochs to keep things fair and fast. With a GPU or more epochs, the one hundred token setting might catch up or even win, especially with gradient clipping and a slightly smaller learning rate. If I had more time, I would also try a small learning rate schedule for SGD and see if that closes the gap.

## Repro note

I trained for exactly ten epochs, batch size thirty-two, embedding size one hundred, two recurrent layers of sixty-four units, binary cross-entropy loss, and CPU only. I changed one thing at a time and recorded accuracy, macro-F1, and average epoch time for each run.